Mini Java Compiler — Phase 01: Lexical Analyzer

CSCI415 – Compiler Design, Fall 2026 | Team: Karim Bassem Joseph, Mahmoud Farag, Ali Essam, Ahmed Shalaby, Omar Walid

The scanner is written from scratch in Python 3.10+ using only the standard library (no Lex/regex/parser libraries). A `SourceCursor` reads the source one character at a time and tracks a 1-indexed line and column. The main loop skips whitespace and comments, then dispatches on the first character: letter or `_` → word scanner, digit → number scanner, quote → string scanner, anything else → operator/delimiter scanner (which reports illegal characters). Identifiers go into a hash-table symbol table (101 buckets, polynomial hash ×31, separate chaining) with every `line:column` where they occur.

1. Implemented Token Specifications (31 token types)

Group	Token type and lexeme / pattern
Class keywords	CLASS `class` · PUBLIC `public` · STATIC `static` · VOID `void` · MAIN `main`
Type keywords	INT `int` · FLOAT `float` · BOOLEAN `boolean` · CHAR `char` · STRING_TYPE `String`
Control keywords	IF `if` · ELSE `else` · WHILE `while` · FOR `for` · RETURN `return`
Names	IDENTIFIER `[A-Za-z_][A-Za-z0-9_]*` that is not a keyword
Literals	NUMBER `[0-9]+(\.[0-9]+)?` (e.g. `10`, `25.5`) · STRING `"..."` or `'...'` on one line, quotes kept in the lexeme, `\` escapes the next character
Operators	ASSIGN `=` · MATHOP `+ - * /` · LOGOP `== != < > <= >=`
Delimiters	LPAR `(` · RPAR `)` · LCUR `{` · RCUR `}` · LSQU `[` · RSQU `]` · SEMICOLON `;` · COMMA `,` · DOT `.`
Control	EOF — emitted once, at the position right after the last character

- **Case-sensitive:** keywords match exactly, so `class` is CLASS but `Class` is IDENTIFIER; `String` is STRING_TYPE while `"text"` is STRING.
- **Longest match:** two-character operators (`== != <= >=`) are tried before one-character ones; words are read completely before the keyword lookup, so `classy` and `integer` are identifiers.
- **Numbers:** a dot belongs to the number only if a digit follows it: `3.` → NUMBER `3` + DOT, and `1.2.3` → NUMBER `1.2`, DOT, NUMBER `3`.
- **Comments and whitespace** (`//` to end of line, `/* ... */`, spaces, tabs, LF, CRLF) produce no tokens. A tab counts as one column.

Sample input

```
class A {
  int x = 10; // counter
  String s = "Hi";
}
```

Output (one line per token, then the symbol table)

```
Lexeme: class, Token: CLASS
Lexeme: A, Token: IDENTIFIER
Lexeme: {, Token: LCUR
Lexeme: int, Token: INT
Lexeme: x, Token: IDENTIFIER
Lexeme: =, Token: ASSIGN
Lexeme: 10, Token: NUMBER
Lexeme: ;, Token: SEMICOLON
Lexeme: String, Token: STRING_TYPE
Lexeme: s, Token: IDENTIFIER
Lexeme: =, Token: ASSIGN
Lexeme: "Hi", Token: STRING
Lexeme: ;, Token: SEMICOLON
Lexeme: }, Token: RCUR
Lexeme: EOF, Token: EOF
Symbol Table
Index: 0, Name: A, Occurrences: 1:7
Index: 1, Name: x, Occurrences: 2:7
Index: 2, Name: s, Occurrences: 3:10
```

2. Error Handling & Recovery Architecture

Error	Exact message format	Skipped before scanning resumes
Illegal character	`Error: invalid character '@' at line L, column C.`	Only that one character.
Invalid identifier	`Error: invalid identifier '1name' at line L, column C.`	The whole word (one error).

Unclosed string	`Error: unclosed string literal starting at line L, column C.`	The rest of that line only.
Unterminated comment	`Error: unclosed comment starting at line L, column C.`	Everything up to end of file.

- **Panic-mode recovery:** each sub-scanner advances past the bad input before returning a `LexicalError` object. The main loop stores it and continues with the next character. Nothing is raised, so scanning never stops early and **all** errors in a file are reported in one run, with every valid token still emitted.
- **Collection and ordering:** tokens and errors are kept in two lists. The formatter merges them by `(line, column)`, so each error is printed exactly where it occurred in the source.
- **Exit codes:** lexical errors are normal output (exit 0, or 1 with `--strict`); a missing file or directory prints one line and exits 1, never a stack trace.

Error example: `int $x = 10 @ 5;` → output:

```
Lexeme: int, Token: INT                          Lexeme: 10, Token: NUMBER
Error: invalid character '$' at line 1, column 5.  Error: invalid character '@' at line 1, column 13.
Lexeme: x, Token: IDENTIFIER                     Lexeme: 5, Token: NUMBER
Lexeme: =, Token: ASSIGN                         Lexeme: ;, Token: SEMICOLON
                                                 Lexeme: EOF, Token: EOF
```

3. Challenges Faced & Solutions

Challenge	How we solved it
CRLF position tracking	Windows files end lines with `\r\n`; counted naively that is two line breaks and every later line number is wrong. `advance()` consumes `\r\n` as one break and resets the column to 1. The same program in LF and CRLF gives identical output.
`/` vs. comments	One character is not enough to decide. The comment skipper looks one character ahead with `peekNext()`: `//` and `/*` start comments, a lone `/` is left for the operator scanner as MATHOP (`5/*c*/6` → NUMBER, NUMBER).
`1name` handling	A naive scanner returns NUMBER `1` then IDENTIFIER `name`. The number scanner checks whether the digits are followed by a letter or `_`; if so it consumes the whole word and reports one invalid-identifier error.
Unclosed string ate the file	The string scanner first read until a closing quote or EOF, hiding every later error. Strings now stop at end of line, so the four required errors can all be reported from one program.
Symbol-table collisions	With 31 buckets and hash multiplier 31, `h*31 % 31 = 0`, so the hash depended only on the last letter (`count`, `limit`, `result` collided). We switched to 101 buckets (a prime different from the multiplier).
Parallel work & merge conflicts	Five people built the scanner at once. Task 1 froze the shared contracts (`TokenType`, `Token`, `LexicalError`, `SourceCursor`) so each member worked on a separate branch and file, merged through reviewed pull requests. Golden expected-output files catch any change to the output format byte for byte.

Testing & Known Limitations

- **Tests:** `tests/test_programs.txt` holds 16 labelled programs (valid code, all tokens, comment edge cases, the four required errors alone and together, empty file, a 1,100-character line). `python tests/verify.py` compares each against `tests/expected/`: **16/16 pass**. Unit tests cover the cursor, scanners, symbol table, CLI and lexer. Extra checks: LF = CRLF output, a 10,000-line file scans in about 0.1 s, empty and whitespace-only files give only EOF.
- **Limitations:** `true`/`false` are scanned as IDENTIFIER (the frozen 31-type enum has no boolean-literal type); `3.14abc` gives NUMBER + IDENTIFIER without an error; non-ASCII letters (e.g. `é`) are accepted in identifiers; `&&` and `||` are not Mini Java operators and are reported as illegal characters.